INSTITUTO TECNOLOGICO DE JIQUILPAN

“Manual del sistema: Stylish Steps Versión 1.0.0”

INGENIERÍA DE SOFTWARE

INGENIERÍA EN SISTEMAS COMPUTACIONALES

PRESENTA:

ISIS LIZBETH VAZQUEZ AVALOS
DANNA GERALDINE ÁVILA GARCÍA
BRYAN GAEL ESQUIVEL PÉREZ

Luis Antonio Díaz Rosales

JIQUILPAN, MICHOACÁN, 20 DE MAYO DE 2025

Contenido

2. Descripción de la arquitectura 3
2.1 Estilo de arquitectura 3
2.2 Componentes principales del sistema 3
2.3 Diagrama general del sistema 6
3. Manual de instalación 14
3.1 Requisitos del sistema 14
3.2 Pasos para la instalación 15
3.3 Configuración inicial 16
4. Estructura del código fuente 16
4.1 Árbol de directorios 16
4.2 Descripción de carpetas y archivos principales 17
4.3 Librerías externas utilizadas 18
5. Base de datos 19
5.1 Diagrama entidad-relación (ER) y modelo relacional 19
5.2 Descripción de las tablas más relevantes y su función 21
5.3 Creación de la base de datos 23
6. Mantenimiento y soporte 29
6.1 Cómo aplicar parches o actualizaciones 29
6.2 Registro de errores conocidos 30
6.3 Contacto para soporte técnico 30

2. Descripción de la arquitectura

2.1 Estilo de arquitectura

La arquitectura del sistema StylishSteps se basa en un modelo multicapa para garantizar modularidad, escalabilidad y seguridad. Se divide en las siguientes capas:

1. Capa de Presentación: interfaz web en React.js, permite la interacción con el usuario.
2. Capa de Lógica de Negocio: backend con Node.js y Express.js, gestiona procesos y reglas.
3. Capa de Acceso a Datos: persistencia de la información en una base de datos MySQL.
4. Capa de Seguridad (opcional): autenticación, autorización y cifrado usando JWT y Bcrypt.

Este modelo permite una separación clara de responsabilidades, favoreciendo el mantenimiento, pruebas y escalabilidad.

2.2 Componentes principales del sistema

1. Capa de Presentación (Frontend)
    - Funciones:
        - El usuario debería poder crear una cuenta y logearse con el sistema.
        - El usuario debería poder visualizar productos
        - El usuario debería poder visualizar el detalle de los productos

- El usuario debería poder visualizar el detalle de un pedido o compra que hizo

• Tecnologías Usadas:

- HTML5, CSS3 y JavaScript para la estructura y estilos.
- React.js para una interfaz interactiva y dinámica.
- Axios para manejar las peticiones HTTP al backend.

2. Capa de Lógica de Negocio (Backend)

• Funciones:

- El usuario debe poder agregar productos que quiera comprar al carrito de compras
- El usuario puede verificar la disponibilidad de los productos antes de hacer su pedido
- El usuario debería poder hacer su pedido de acuerdo a los artículos de su carrito

• Tecnologías Usadas:

- Node.js con Express.js para un backend ligero y escalable.

3. Capa de Datos (Base de Datos)

• Funciones:

- El sistema podrá buscar en la base de datos a clientes o administradores
- (según corresponda), con las credenciales establecidas para autentificarlos
- con el sistema y puedan entrar.

• Tecnologías Usadas:

- MySQL o PostgreSQL para bases de datos relacionales.

4. Capa de Seguridad y Autenticación (Opcional)
    - Funciones:
        - Autenticación y gestión de sesiones (inicio/cierre de sesión).
        - Cifrado de contraseñas y datos sensibles.
        - Implementación de roles de usuario (cliente, administrador).
        - Integración con métodos de autenticación externos (Google, Facebook, etc.).
    - Tecnologías Usadas:
        - JWT (JSON Web Token) para autenticación segura.
        - OAuth 2.0 para login con cuentas externas.
        - Bcrypt o Argon2 para cifrado de contraseñas.
        - HTTPS y certificados SSL para comunicaciones seguras.

2.3 Diagrama general del sistema

A continuación, se representa la comunicación entre las capas principales con un diagrama representativo:

Diagrama de Arquitectura de capas:

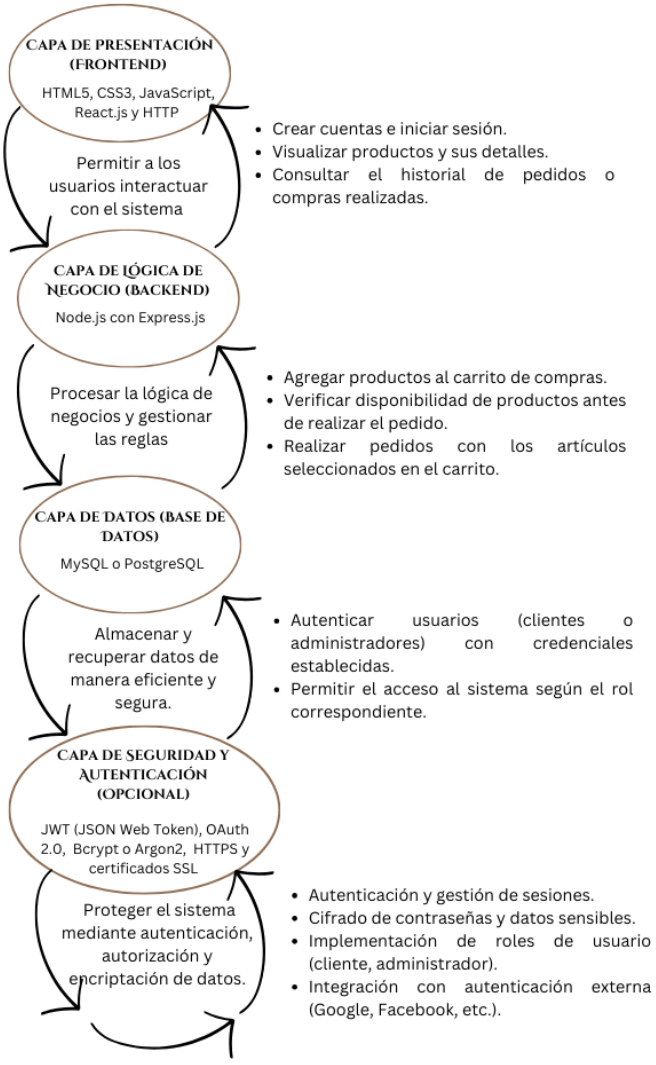


Diagrama de clase de Stylish Steps:

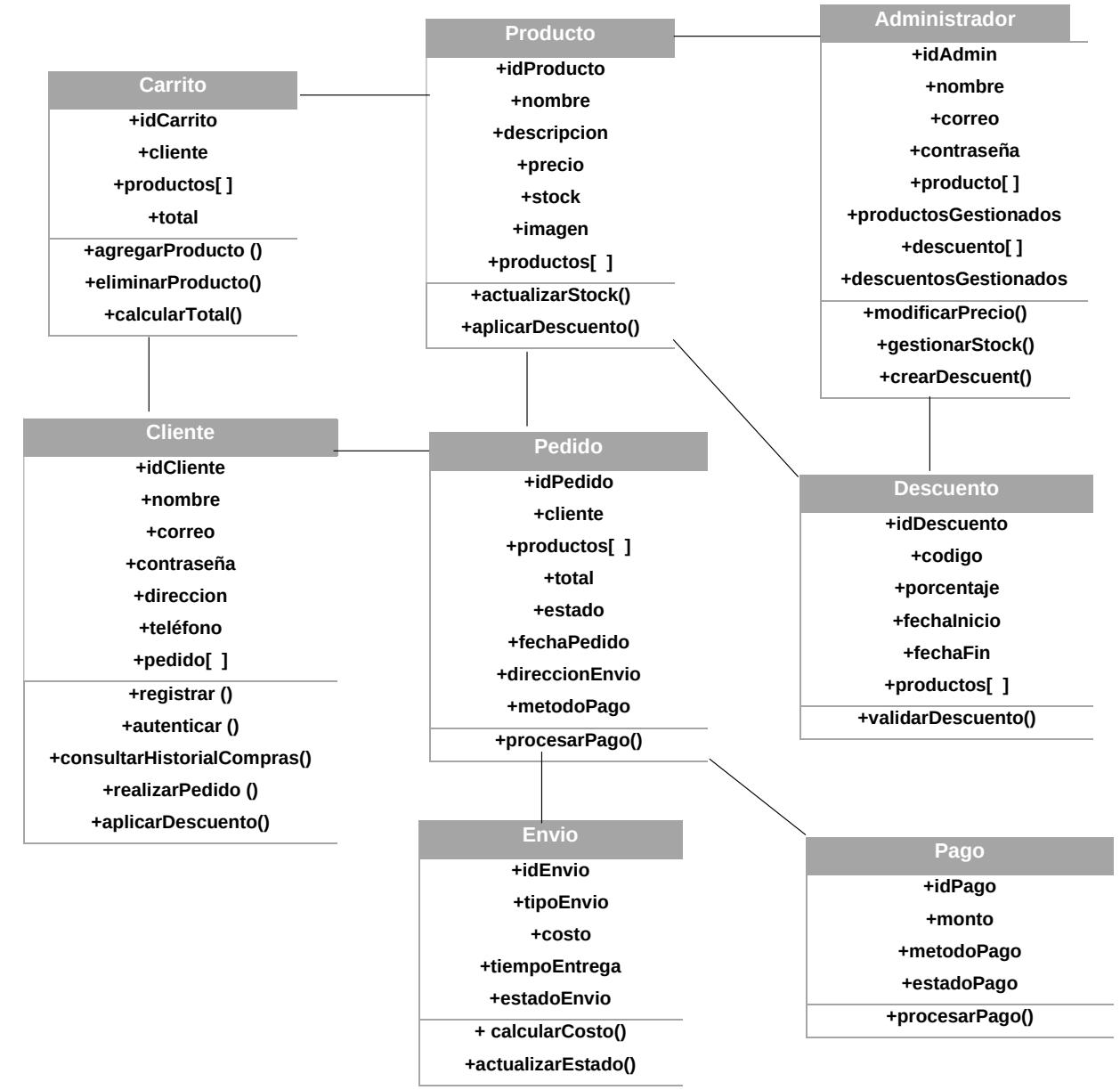


Diagrama de Caso de Uso:

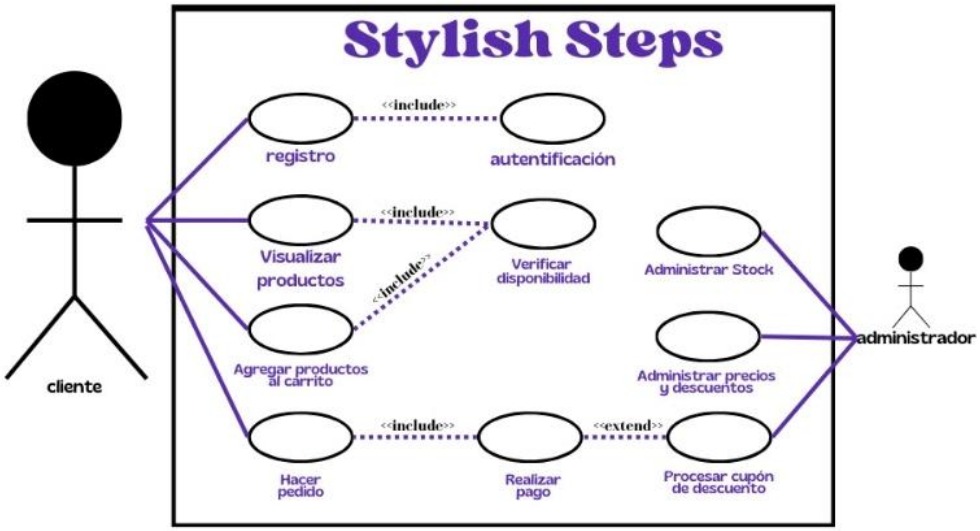


Diagrama de causa y efecto:



Diagrama del modelo entidad relación (E-R):

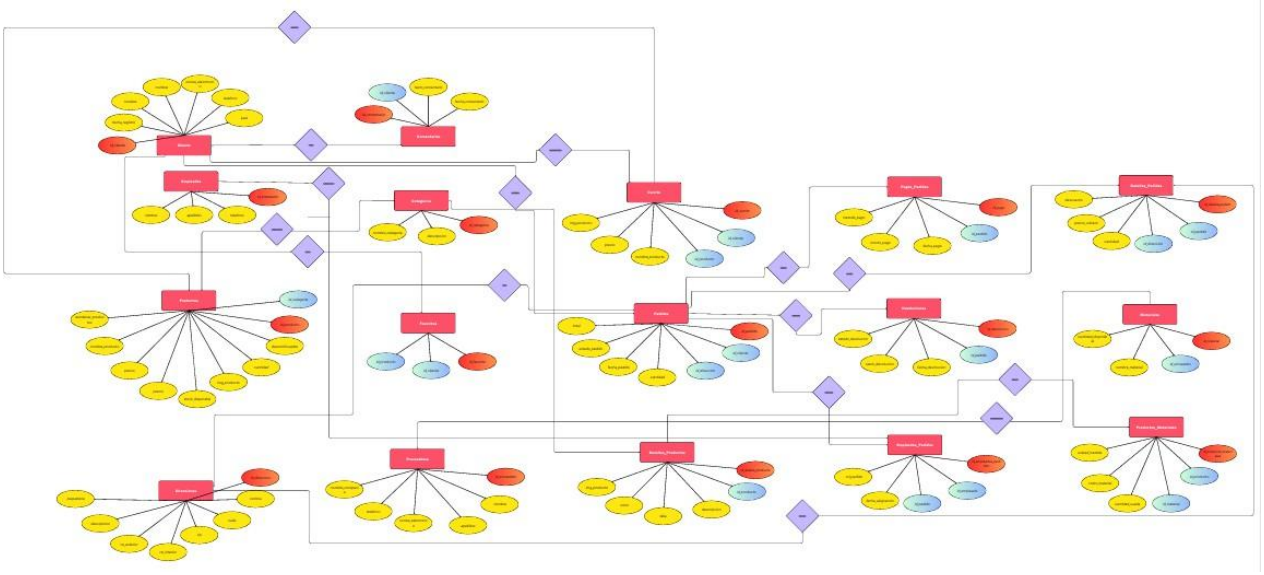


Diagrama del modelo relación:

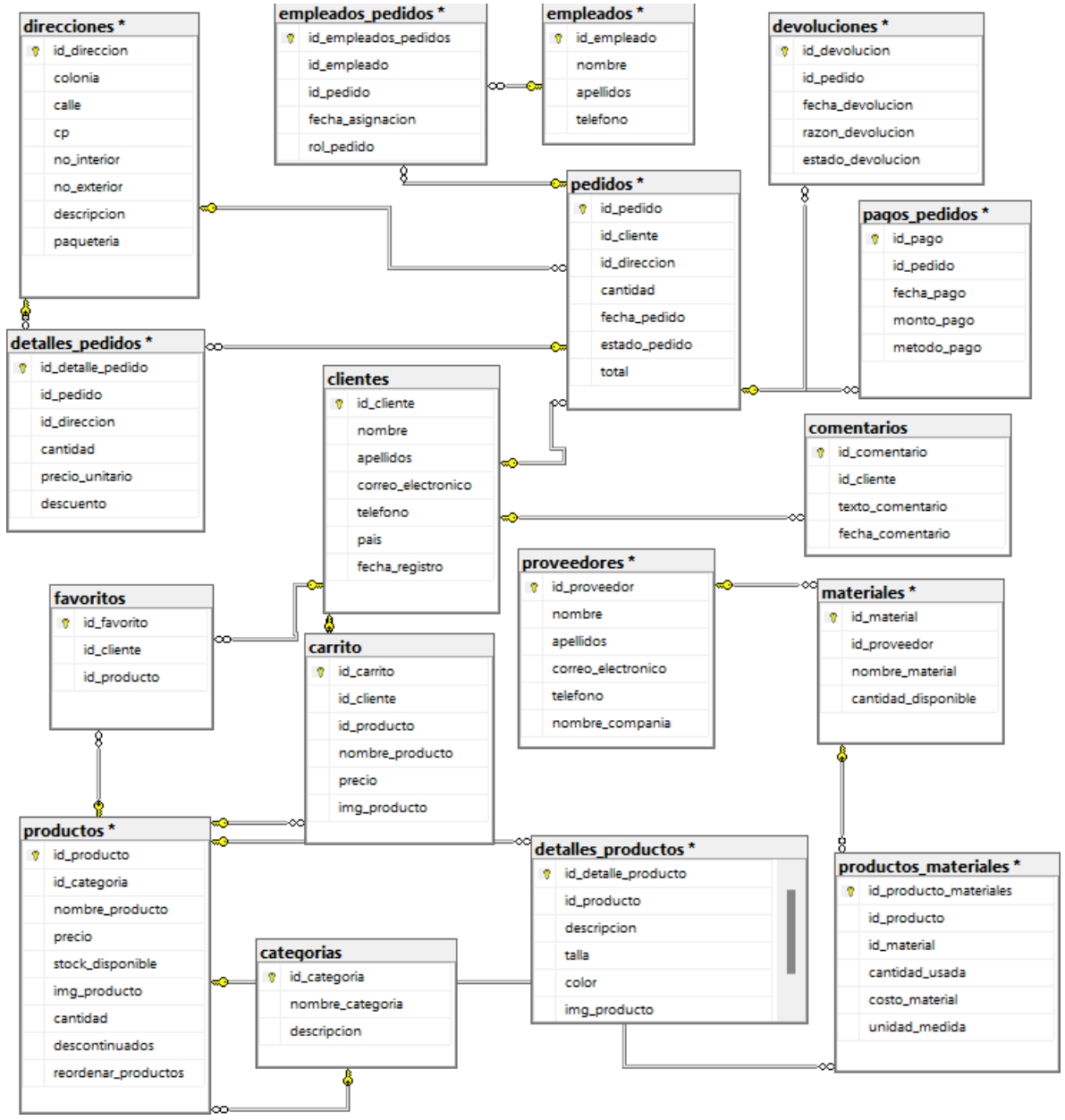


Diagrama de flujo de registro y autentificación:

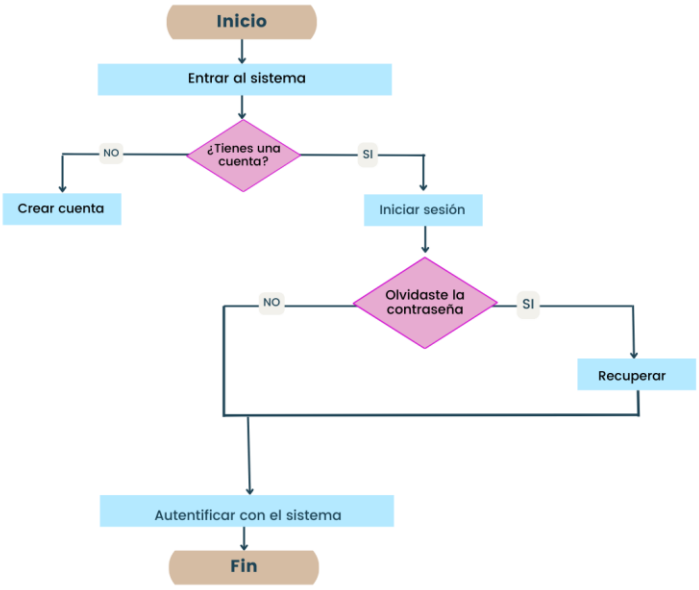


Diagrama de flujo de hacer pedido y hacer pago:

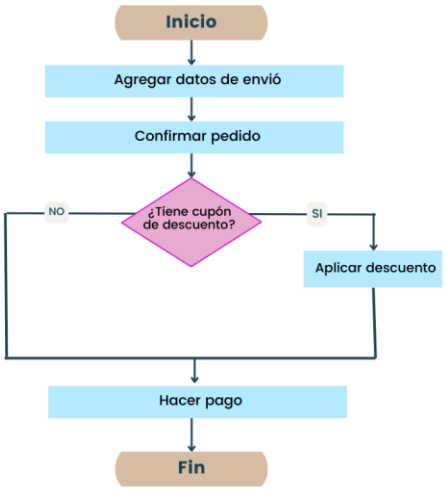


Diagrama de flujo de visualizar producto y agregarlos al carrito:

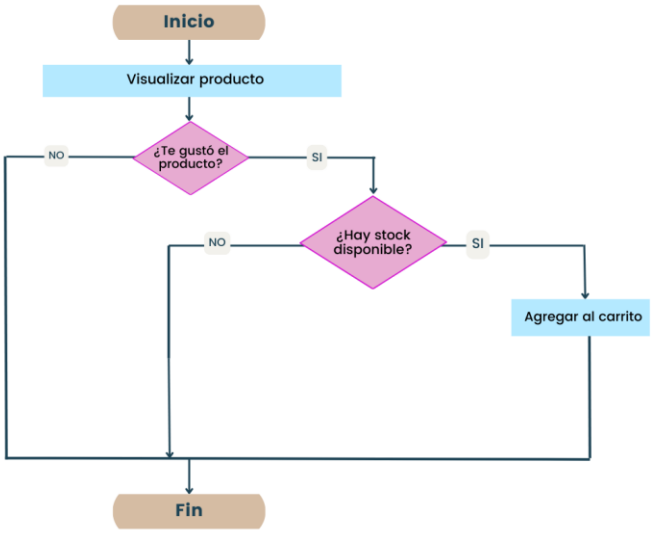
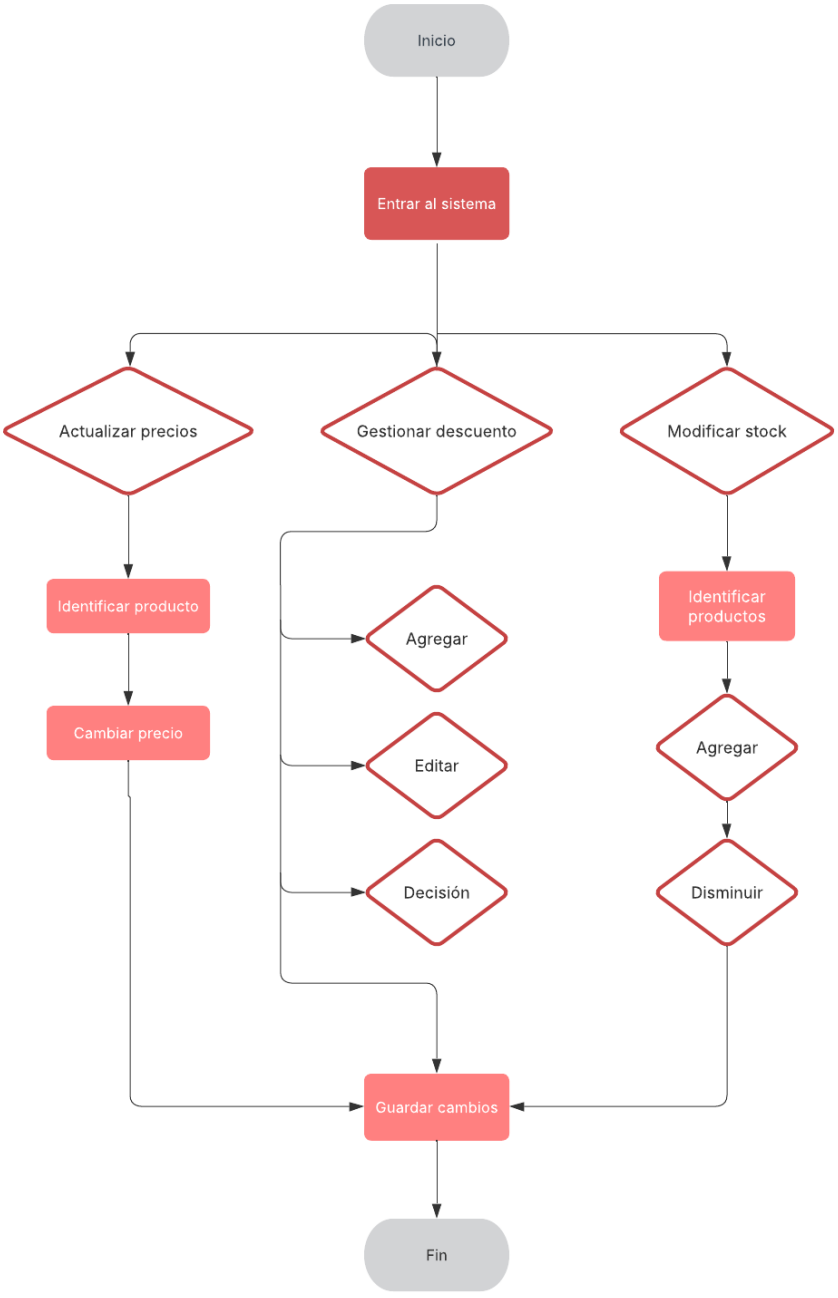


Diagrama de flujo de administrar el stock, el descuento y el precio:



2.4 Flujo de datos o interacción

A continuación, se describe cómo fluye la información en el sistema Stylish Steps:

1. El usuario accede a la tienda desde su navegador y visualiza los productos.
2. Puede buscar, filtrar y seleccionar artículos.
3. Agrega productos al carrito y procede al pago.
4. Si no ha iniciado sesión, se le pide autenticarse o registrarse.
5. El frontend envía peticiones HTTP al backend para validar credenciales o procesar la compra.
6. El backend consulta la base de datos para verificar datos, stock y precios.
7. Se genera el pedido, se registra en la base de datos y se envía una confirmación por correo electrónico.
8. La pasarela de pago valida la transacción.
9. El usuario recibe confirmación visual en la interfaz y su pedido queda registrado.

3. Manual de instalación

3.1 Requisitos del sistema

- **Sistema Operativo:** Windows 10+ / Ubuntu 20.04+
- **Procesador:** Intel Core i5
- **RAM:** 4 GB mínimo
- **Disco:** 1 GB libre
- **Dependencias:**
    - Node.js v18+
    - MySQL 8.0

- Navegador (Chrome o Firefox)

3.2 Pasos para la instalación

Este apartado explica paso a paso cómo instalar el sistema completo en un entorno local de desarrollo.

1. Clonar el repositorio del sistema desde GitHub:
   - git clone https://github.com/usuario/stylish-steps.git
2. Acceder al directorio del proyecto:
   - cd stylish-steps
3. Instalar las dependencias del backend y frontend:
   - npm install
4. Configurar el archivo .env con las siguientes variables:
   - DB_HOST=localhost
   - DB_USER=root
   - DB_PASSWORD=tu_contraseña
   - DB_NAME=stylish_db
   - PORT=3000
5. Crear la base de datos ejecutando el script proporcionado:
   - mysql -u root -p < db/init.sql
6. Iniciar el servidor:
   - npm start
7. Acceder a la aplicación desde el navegador:
   - http://localhost:3000

3.3 Configuración inicial

Después de la instalación, se deben seguir estos pasos para dejar el sistema listo para ser usado:

- Inicio de sesión:
  - Usuario inicial: admin
  - Contraseña: admin123
- Cambio de contraseña:
  - Ingresar al panel de usuario
  - Cambiar la contraseña desde el menú de perfil
- Configuración de parámetros del sistema:
  - Idioma
  - Zona horaria
  - Moneda (ej. MXN)
- Verificación de funcionamiento:
  - Acceder a la página de productos
  - Probar agregar un producto al carrito
  - Simular una compra para probar el flujo de ventas

4. Estructura del código fuente

4.1 Árbol de directorios

```
/huarachesShop
|
├── /src
```

```
│   ├── /controllers
│   │   └── productController.js
│   ├── /models
│   │   └── productModel.js
│   ├── /views
│   │   └── productView.jsx
│   ├── main.js
│
├── /config
│   └── database.js
│
├── /public
│   ├── /css
│   │   └── style.css
│   ├── /js
│   │   └── script.js
│
├── /routes
│   └── productRoutes.js
│
├── /middlewares
│   └── authMiddleware.js
│
└── README.md
```

4.2 Descripción de carpetas y archivos principales

- **/controllers/**: Contiene la lógica del negocio, como la gestión de productos, usuarios, pedidos, etc.
    - Ejemplo: productController.js gestiona las operaciones sobre los productos como listar, agregar o eliminar huaraches.
- **/models/**: Define la estructura de los datos y su conexión con la base de datos.
    - Ejemplo: productModel.js define los atributos del producto como nombre, talla, precio y categoría.
- **/views/**: Contiene las plantillas visuales del sistema (HTML, JSX, etc.).
    - Ejemplo: productView.jsx muestra los productos al usuario en la interfaz de la tienda.
- **main.js**: Punto de entrada del sistema. Se encarga de iniciar el servidor y cargar rutas y middlewares.
- **/config/database.js**: Archivo donde se establece la conexión con la base de datos (por ejemplo, MongoDB o MySQL).
- **/public/**: Contiene archivos estáticos como hojas de estilo (/css/) y scripts (/js/).
- **/routes/**: Define las rutas del sistema, separando la lógica de acceso a cada controlador.
    - Ejemplo: productRoutes.js contiene las rutas de productos (GET, POST, DELETE...).
- **/middlewares/**: Contiene funciones intermedias como autenticación, validaciones, manejo de errores, etc.

4.3 Librerías externas utilizadas

- **Express.js**: Framework para crear el servidor web y manejar las rutas.
- **Sequelize**: ORM/ODM para conectarse a SQL Server.

- **React.js**: Librería de frontend usada en las vistas (opcional).
- **dotenv**: Para manejo de variables de entorno.
- **bcryptjs**: Para encriptar contraseñas.
- **jsonwebtoken**: Para manejar la autenticación con tokens.
- **Cors**: Para permitir peticiones entre dominios.
- **Nodemon**: Para reiniciar el servidor automáticamente en desarrollo.

5. Base de datos

5.1 Diagrama entidad-relación (ER) y modelo relacional

Modelo entidad-relación (E-R)

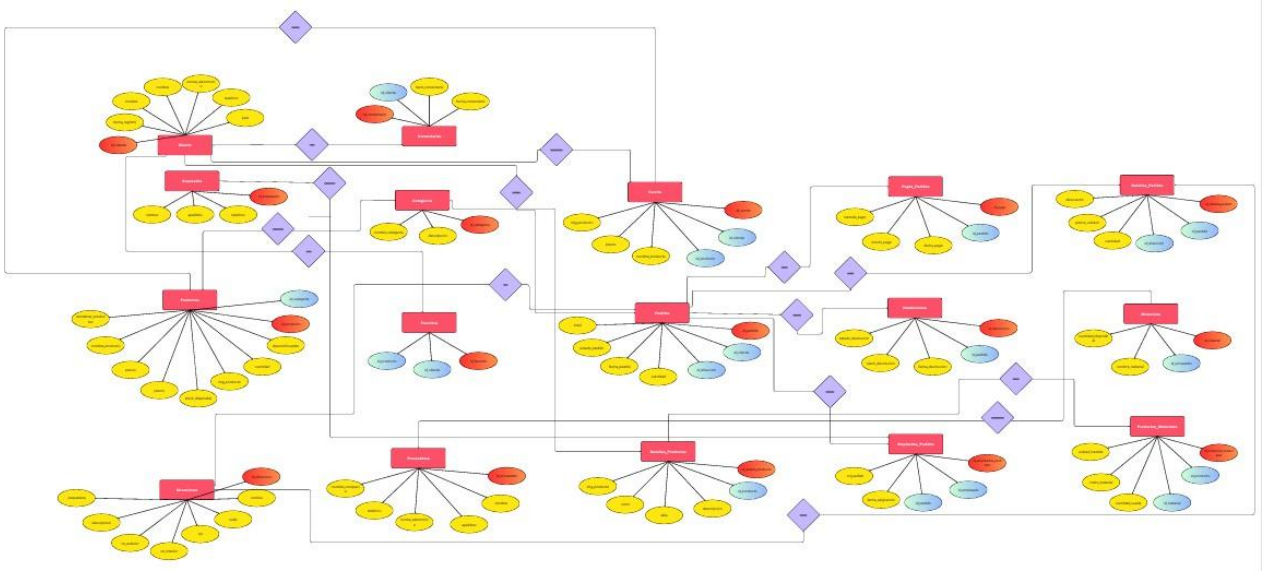
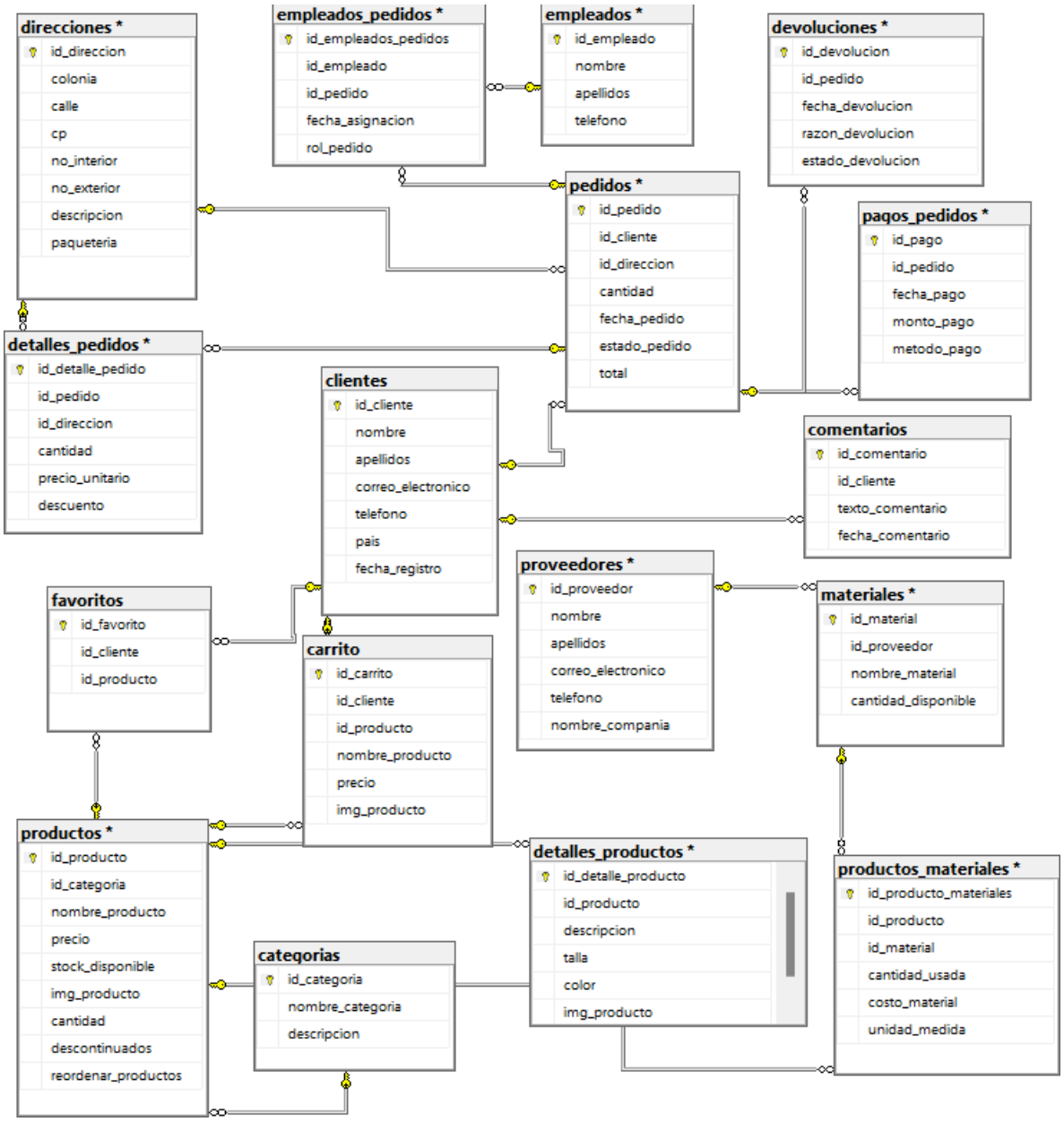


Diagrama del modelo relación



5.2 Descripción de las tablas más relevantes y su función

Tablas de Gestión de Usuarios y Clientes:

- Clientes: Almacena información de los compradores incluyendo nombre, apellidos, correo electrónico, teléfono y país. Funciona como base para la gestión de relaciones con los clientes.
- Direcciones: Guarda las direcciones de envío de los clientes con información detallada como colonia, calle, código postal y especificaciones de entrega.

Tablas de Gestión de Productos:

- Productos: Registra los artículos disponibles para venta con su nombre, precio, stock disponible e imagen. Es la tabla central para el catálogo de productos.
- Categorías: Clasifica los productos en diferentes tipos permitiendo una organización jerárquica del catálogo.
- Detalles_Productos: Contiene información específica sobre cada producto como talla, color y descripciones detalladas.

Tablas de Gestión de Inventario y Materiales:

- Materiales: Almacena información sobre los insumos utilizados en la fabricación de los huaraches, incluyendo cantidad disponible y proveedor.
- Productos_Materiales: Relaciona productos con los materiales utilizados en su fabricación, registrando la cantidad usada y el costo de cada material por producto.
- Proveedores: Guarda información de las empresas o personas que suministran materiales a la tienda.

Tablas de Gestión de Ventas:

- Pedidos: Registra las órdenes de compra realizadas por los clientes, incluyendo fecha, estado del pedido y total.
- Detalles_Pedidos: Almacena información específica sobre cada producto incluido en un pedido, como cantidad y precio unitario.
- Pagos_Pedidos: Registra la información de los pagos asociados a cada pedido, incluyendo método de pago y monto.

Tablas de Interacción con Clientes:

- Carrito: Almacena temporalmente los productos que los clientes agregan a su carrito antes de finalizar la compra.
- Favoritos: Registra los productos marcados como favoritos por los clientes, permitiendo personalizar la experiencia de usuario.
- Comentarios: Guarda opiniones y reseñas dejadas por los clientes sobre productos o servicios.

Tablas de Gestión Operativa:

- Empleados: Contiene información sobre el personal de la tienda.
- Empleados_Pedidos: Relaciona a los empleados con los pedidos que atienden, registrando su rol y fecha de asignación.
- Devoluciones: Almacena información sobre productos devueltos, incluyendo la razón y el estado de la devolución.

5.3 Creación de la base de datos

- Detalles_Productos

```
create table detalles_productos(
---------------------LLAVES-------------------------
    id_detalle_producto integer identity(1,1) primary key,
    id_producto integer not null foreign key(id_producto) references productos(id_producto),
----------------------------------------------------
    descripcion varchar(50) not null,
    talla integer not null,
    color varchar(20) not null,
    img_producto varbinary(max) not null
);
```

- Pagos-Pedidos

```
create table pagos_pedidos(
----------------------LLAVES-------------------------
    id_pago integer identity(1,1) primary key,
    id_pedido integer not null foreign key(id_pedido) references pedidos(id_pedido),
----------------------------------------------------
    fecha_pago date not null,
    monto_pago integer not null,
    metodo_pago varchar(30) not null
);
```

- Devoluciones

```
create table devoluciones(
----------------------LLAVES-------------------------
    id_devolucion integer identity(1,1) primary key,
    id_pedido integer not null foreign key(id_pedido) references pedidos(id_pedido),
----------------------------------------------------
    fecha_devolucion date not null,
    razon_devolucion varchar(100) not null,
    estado_devolucion varchar(30) not null
);
```

- Empleados_Pedidos

```
create table empleados_pedidos(
--------------------LLAVES-------------------------
    id_empleados_pedidos integer identity(1,1) primary key,
    id_empleado integer not null foreign key(id_empleado) references empleados(id_empleado),
    id_pedido integer not null foreign key(id_pedido) references pedidos(id_pedido),
---------------------------------------------------
    fecha_asignacion date not null,
    rol_pedido varchar(100) not null,
);
```

- Detalles_Pedidos

```
create table detalles_pedidos(
--------------------LLAVES-------------------------
    id_detalle_pedido integer identity(1,1) primary key,
    id_pedido integer not null foreign key(id_pedido) references pedidos(id_pedido),
    id_direccion integer not null foreign key(id_direccion) references direcciones(id_direccion),
---------------------------------------------------
    cantidad integer not null,
    precio_unitario integer not null,
    descuento integer
);
```

- Materiales

```
create table materiales(
--------------------LLAVES-------------------------
    id_material integer identity(1,1) primary key,
    id_proveedor integer not null foreign key(id_proveedor) references proveedores(id_proveedor),
---------------------------------------------------
    nombre_material varchar(40) not null,
    cantidad_disponible integer not null,
);
```

- Productos_Materiales

```
create table productos_materiales(
--------------------LLAVES-------------------------
    id_producto_materiales integer identity(1,1) primary key,
    id_producto integer not null foreign key(id_producto) references productos(id_producto),
    id_material integer not null foreign key(id_material) references materiales(id_material),
---------------------------------------------------
    cantidad_usada integer not null,
    costo_material integer not null,
    unidad_medida varchar(50),
);
```

- Clientes

```
create table clientes(
----------------------LLAVES--------------------------
    id_cliente integer identity(1,1) primary key,
-------------------------------------------------------
    nombre varchar(30) not null,
    apellidos varchar(50) not null,
    correo_electronico varchar(80) not null,
    telefono varchar(10) not null,
    pais varchar(30),
    fecha_registro date not null
);
```

- Comentarios

```
create table comentarios(
----------------------LLAVES--------------------------
    id_comentario integer identity(1,1) primary key,
    id_cliente integer not null foreign key(id_cliente) references clientes(id_cliente),
-------------------------------------------------------
    texto_comentario varchar(100) not null,
    fecha_comentario date not null
);
```

- Empleados

```
create table empleados(
----------------------LLAVES--------------------------
    id_empleado integer identity(1,1) primary key,
------------------------------------------------------
    nombre varchar(30) not null,
    apellidos varchar(50) not null,
    telefono integer not null,
);
```

- Categorías

```
create table categorias(
----------------------LLAVES--------------------------
    id_categoria integer identity(1,1) primary key,
------------------------------------------------------
    nombre_categoria varchar(30) not null,
    descripcion varchar(50) not null,
    --stock_categoria integer not null
);
```

- Productos

```
create table productos(
----------------------LLAVES--------------------------
    id_producto integer identity(1,1) primary key,
    id_categoria integer not null foreign key(id_categoria) references categorias(id_categoria),
------------------------------------------------------
    nombre_producto varchar(50) not null,
    precio integer not null,
    stock_disponible integer not null,
    img_producto varbinary(max) not null,
    cantidad integer not null,
    descontinuados integer not null,
    reordenar_productos integer not null
);
```

- Favoritos

```
create table favoritos(
--------------------LLAVES-------------------------
    id_favorito integer identity(1,1) primary key,
    id_cliente integer not null foreign key(id_cliente) references clientes(id_cliente),
    id_producto integer not null foreign key(id_producto) references productos(id_producto),
----------------------------------------------------
);
```

- Direcciones

```
create table direcciones(
---------------------LLAVES--------------------------
    id_direccion integer identity(1,1) primary key,
-----------------------------------------------------
    colonia varchar(50) not null,
    calle varchar(50) not null,
    cp integer not null,
    no_interior integer not null,
    no_exterior integer not null,
    descripcion varchar(100) not null,
    paqueteria varchar(50) ,
);
```

- Proveedores

```
create table proveedores(
---------------------LLAVES--------------------------
    id_proveedor integer identity(1,1) primary key,
-----------------------------------------------------
    nombre varchar(30) not null,
    apellidos varchar(50) not null,
    correo_electronico varchar(80) not null,
    telefono varchar(10) not null,
    nombre_compania varchar(50) not null,
);
```

- Carrito

```
create table carrito(
----------------------LLAVES--------------------------
    id_carrito integer identity(1,1) primary key,
    id_cliente integer not null foreign key(id_cliente) references clientes(id_cliente),
    id_producto integer not null foreign key(id_producto) references productos(id_producto),
-------------------------------------------------------
    nombre_producto varchar(50) not null,
    precio integer not null,
    --stock_disponible integer not null,
    img_producto varbinary(max) not null,
);
```

- Pedidos

```
create table pedidos(
----------------------LLAVES--------------------------
    id_pedido integer identity(1,1) primary key,
    id_cliente integer not null foreign key(id_cliente) references clientes(id_cliente),
    id_direccion integer not null foreign key(id_direccion) references direcciones(id_direccion),
-------------------------------------------------------
    --descuento integer,
    cantidad integer not null,
    fecha_pedido date not null,
    estado_pedido varchar(20),
    total integer not null,
    --direccion_envio varchar(50)
);
```

6. Mantenimiento y soporte

Este apartado proporciona lineamientos para mantener el sistema actualizado, documentar errores conocidos y establecer un canal de soporte técnico para usuarios y desarrolladores.

6.1 Cómo aplicar parches o actualizaciones

Para mantener Stylish Steps actualizado y funcional, se recomienda seguir este procedimiento cuando haya una nueva versión del sistema:

Procedimiento general de actualización:

1. Realizar respaldo completo:
    - Exportar la base de datos actual (ej. mysqldump).
    - Copiar los archivos de configuración, como .env.
2. Descargar la última versión del repositorio:
    - git pull origin main
3. Instalar nuevas dependencias si las hubiera:
    - npm install
4. Ejecutar scripts de migración (si cambia la base de datos):
    - npm run migrate
5. Reiniciar el sistema:
    - npm restart
6. Verificar funcionamiento:
    - Comprobar rutas principales (/login, /productos, /carrito)
    - Confirmar que el frontend carga correctamente y se conecta al backend.

6.2 Registro de errores conocidos

A continuación, se presenta una tabla de errores detectados durante el desarrollo y su estado actual. Este registro permite hacer seguimiento y priorizar correcciones.

ID	Descripción del error	Versión	Estado	Fecha de reporte
E01	El botón "Finalizar compra" no responde en móviles	v1.0.0	En revisión	08/05/2025
E02	Problemas al recuperar contraseña desde Gmail	v1.0.1	Resuelto	10/05/2025
E03	La carga de productos falla si no hay imágenes	v1.1.0	Reportado	12/05/2025

6.3 Contacto para soporte técnico

Para reportar errores, solicitar soporte o colaborar en mejoras, se puede contactar al responsable técnico del sistema.

- Responsable: Equipo Stylish Steps – Área de Desarrollo Web
- Correo electrónico: soporte@stylishsteps.com
- Teléfono: +52 55 1234 5678
- Horario de atención: Lunes a viernes, 9:00 a 17:00 h
- Documentación técnica oficial: disponible en el repositorio /docs del proyecto